# Achernar

## End-to-End Wiring Audit, Fixes & Roadmap

Unified Astronomy Imaging and Control System

| | |
|---|---|
| **Scope** | Daemon ↔ Backend ↔ Frontend ↔ Desktop wiring |
| **Repository** | `Achernar.tech` (monorepo) |
| **Reference spec** | `achernar_v4` (System Architecture, rev. Mar 2026) |
| **Prepared for** | Ibrahim Shahid, Lead Architect |
| **Date** | June 13, 2026 |
| **Document type** | Engineering audit + change record |

This report documents the current state of the four-tier system, the end-to-end wiring fixes applied in this engineering pass, the verification performed, and a prioritized roadmap toward a working application.

# Contents

**1 Executive Summary** **2**

**2 System Architecture** **2**
2.1 The four tiers . . . 2
2.2 End-to-end data flow . . . 3

**3 Per-Tier Audit** **3**
3.1 Daemon and the C/C++ FFI bridge . . . 3
3.2 Backend (Django control plane) . . . 4
3.3 Frontend (React/TypeScript) . . . 4
3.4 Desktop (Tauri shell) . . . 4

**4 End-to-End Wiring Analysis** **5**
4.1 Browser path (transport = django) . . . 5
4.2 Desktop path (transport = daemon) . . . 5
4.3 Daemon reverse tunnel (cloud relay) . . . 5

**5 Issues Found and Fixes Implemented** **5**
5.1 Fix 1 — Cache backend for the relay handoff (**High**) . . . 6
5.2 Fix 2 — Consolidate the frontend API layer (**High**) . . . 6
5.3 Fix 3 — Track the frontend `src/lib` directory (**High**) . . . 6
5.4 Fix 4 — Tauri hosts the web app in development (**Med**) . . . 6
5.5 Fix 5 — Product naming (**Low**) . . . 7

**6 Verification** **7**

**7 Roadmap to a Working App** **7**

# 1 Executive Summary

Achernar is a four-tier system for controlling astronomy imaging equipment. The repository is substantially more mature than a prototype: all four tiers exist, the hardware-access core compiles to a binary, and the architecture closely tracks the `achernar_v4` specification. The dominant problems are not missing components — they are *wiring* defects: places where two tiers were independently correct but could not reliably talk to each other, and an unfinished refactor that left the frontend with two competing copies of its API layer.

This pass focused on the **end-to-end wiring** between the tiers. We mapped every inter-tier connection (HTTP routes, WebSocket channels, the cloud reverse-tunnel relay, and the runtime transport switch), audited build/health status, and corrected five concrete defects — including two that silently break the system only in production or only on a fresh clone, which is exactly the class of bug that is most expensive to discover later.

| Tier | Status | Headline |
|---|---|---|
| Rust daemon + C/C++ FFI bridge | **Healthy** | Compiles to binary; FFI surface is sound |
| Django backend (17 apps) | **Healthy** | Relay correct after cache fix |
| React/TS web frontend | **Healthy** | Split-brain API layer consolidated |
| Tauri desktop shell | **Partial** | Sidecar mgr solid; dev wiring fixed |
| OpenLiveStacker drivers / SDKs | **Bloated** | Vendored twice (≈9.6 GB) |

Table 1: Per-tier status at the end of this pass.

**Five fixes shipped this pass** (Section 5): (1) a Redis-backed cache so the command relay survives multi-process deployment; (2) consolidation of the duplicated frontend API/env/hooks layers; (3) a `.gitignore` correction so the frontend's transport-switch code is actually tracked; (4) a Tauri `beforeDevCommand` so the desktop hosts the real web app in development; and (5) product naming/title cleanup. All changes were verified (Section 6).

# 2 System Architecture

## 2.1 The four tiers

- **Daemon** (`achernar-daemon`, Rust, ≈5,800 LOC across 11 subsystems): the on-site edge service. Exposes an HTTP+WebSocket API on `127.0.0.1:3030` and talks to hardware through a C/C++ FFI bridge into the OpenLiveStacker (OLS) drivers and vendor SDKs.
- **Backend** (`backend`, Django 4.2 + DRF + Channels, ≈2,500 LOC, 17 apps): the cloud control plane. Hosts the REST API the browser calls and the WebSocket "gateway" the daemon dials into (the IoT reverse tunnel).
- **Frontend** (`achernar-frontend`, React 18 + TypeScript + Vite, ≈1,600 LOC): the operator UI. At runtime it chooses one of two transports — direct-to-daemon (desktop) or via the Django relay (browser).
- **Desktop** (`achernar-desktop`, Tauri 2, ≈285 LOC Rust): a thin shell that *hosts the web frontend* and *manages the daemon as a sidecar* (launch / health-check / stop).

### 2.2 End-to-end data flow

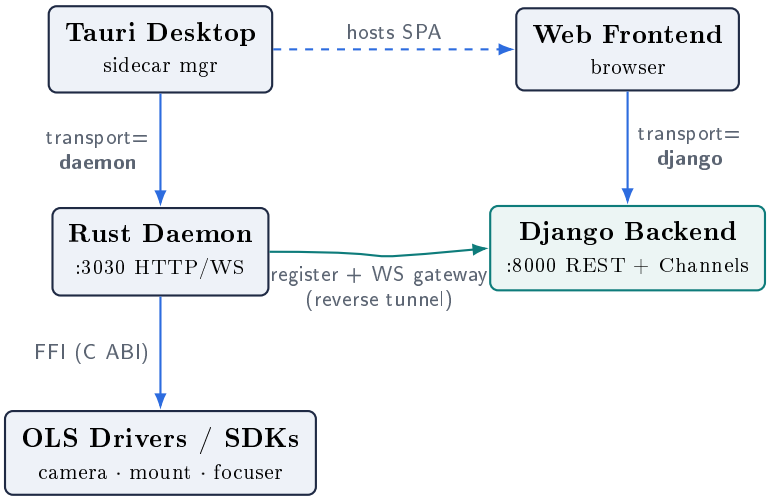



Figure 1: Two operator paths converge on one daemon. In the desktop, the frontend detects the Tauri runtime and talks *directly* to the local daemon; in the browser it talks to Django, which relays commands to the daemon over a daemon-initiated WebSocket "gateway."

The crucial design idea (Figure 1) is the **runtime transport switch** (`src/lib/runtime.ts`): `getHardwareTransport()` returns `"daemon"` inside Tauri and `"django"` in a browser. Every data hook keys off this, so the same React code drives both the desktop (LAN-local, low latency) and the browser (internet, via relay).

## 3 Per-Tier Audit

### 3.1 Daemon and the C/C++ FFI bridge

The hardware-access core is the part you specifically asked to validate, and it is in good shape. The Rust source is organised into the subsystems the spec calls for:

```
ai/  api/  control/  filter_wheel/  focus/  hardware/
models/  mount/  processing/  storage/  vision/
control/{control_supervisor, action_arbiter, safety_guard, ownership}.rs
focus/{focus_agent, focus_inference, focus_policy, backlash, ...}.rs
```

**FFI appropriateness (your headers).** The bridge is a clean three-layer design:

1. `ols-bridge/include/ols_bridge.h` — a *flat C ABI* (opaque handles + integer error codes) wrapping the OLS C++ classes. This is the correct boundary: a stable `extern "C"` surface is what Rust can bind to safely.
2. `src/hardware/ols_ffi.rs` — Rust `extern` declarations that map 1:1 to the header, with `#[repr(C)]` structs for frames, formats and parameters.
3. `ols-bridge/src/ols_bridge.cpp` — the implementation, compiling OLS driver sources directly and replacing the heavy `cppcms`/`booster` dependencies with header-only shims (`nlohmann/json` + minimal socket wrappers).

The C symbol surface covers camera (sim / Alpaca / ASI / SVBony / Touptek), mount (Alpaca), and focuser (Alpaca) with create / list / open / parameter / stream entry points, and `build.rs` drives the CMake build and links the result. **Evidence it compiles:** both

`target/debug/achernar-daemon` and `target/release/achernar-daemon` are present as built binaries. We did not force a full `cargo` rebuild in this pass (it is heavy — LTO plus ≈428 C++ translation units plus an ONNX Runtime download) and we made *no* Rust changes, so the existing binaries remain valid.

| Daemon HTTP/WS route | Purpose |
|---|---|
| `GET /health` | Liveness (used by the Tauri sidecar manager) |
| `GET /api/v1/status` | Aggregate camera/mount/focuser state |
| `POST /api/v1/connect` | Connect equipment by `{type, driver, option}` |
| `POST /api/v1/camera/stream/start` | Begin live view |
| `GET /api/v1/camera/stream` (WS) | Binary frame packets |
| `POST /api/v1/mount/*` | slew / sync / park / tracking / abort |
| `POST /api/v1/focuser/*` | move / moveto / position / abort |
| `GET /ws/telemetry` (WS) | 1 Hz telemetry stream |

Table 2: Daemon API surface actually mounted in `main.rs`.

**Gaps (roadmap, not wiring):** `api/focus.rs` defines the spec's autofocus control endpoints (`/api/v1/focus/*`) but every handler is a `todo!()` stub; they are correctly *not* mounted (mounting panicking stubs would be worse). The frontend's `analyze()` helper points at `/api/v1/ai/analyze`, which the daemon does not serve; it is unused by the live UI.

### 3.2 Backend (Django control plane)

Seventeen apps, ASGI-first (`daphne` is the first installed app), Channels for WebSockets, DRF for REST. The reverse-tunnel relay is implemented end-to-end: `apps/sync/views.py` registers daemons, `apps/realtime/consumers.py` runs the `DaemonConsumer` (gateway) and `TelemetryConsumer` (browser fan-out), and `apps/equipment/views.py` turns a browser REST call into a relayed command and waits for the result. Dev/prod settings are cleanly split; dev falls back to SQLite + in-memory Channels, prod mandates Postgres + Redis. The one real defect here (the cache handoff) is detailed in Section 5.

### 3.3 Frontend (React/TypeScript)

Feature-organised (`cloud, filter, focus, imaging, mount, session, settings`) with a clean `api / hooks / pages / stores / types` split and React Query for data. The transport switch is the right abstraction. The defect was an **unfinished `src/` → `src/lib/` migration** that left two divergent copies of the API client, the env config, and two hooks — with `Dashboard.tsx` importing from *both*. Consolidated this pass (Section 5).

### 3.4 Desktop (Tauri shell)

The Rust side is the highlight: `src-tauri/src/lib.rs` is a genuine `DaemonManager` that locates the daemon binary, launches it as a child process, performs a raw-socket health check, forwards cloud env vars, and tears the process down on exit. Configuration already points the bundled app at the real web frontend (`frontendDist`) and builds it (`beforeBuildCommand`). What was

missing was the *development* counterpart — nothing started the frontend dev server — plus leftover scaffold (the default "Tauri + React" greet page) that is dead because the shell hosts the frontend, not its own `src/`.

# 4 End-to-End Wiring Analysis

## 4.1 Browser path (transport = django)

```
Browser  --HTTP-->  Django /api/equipment/command/  (send_relay_command)
         channel_layer.group_send -> DaemonConsumer (WS) -> daemon
daemon executes -> COMMAND_RESULT -> DaemonConsumer -> cache.set(result)
Django view polls cache.get(result)  ----> HTTP response to browser
```

This loop is correct *provided the WS consumer and the HTTP view share one cache* — the subject of Fix 1.

## 4.2 Desktop path (transport = daemon)

```
Tauri launches -> DaemonManager.start() spawns achernar-daemon (:3030)
Webview loads frontend -> getHardwareTransport()=="daemon"
Frontend --HTTP/WS--> daemon directly (status, stream, commands)
```

Correct, and now complete in dev once Fix 4 starts the frontend server that the webview points at.

## 4.3 Daemon reverse tunnel (cloud relay)

```
daemon -> POST {ACHERNAR_CLOUD_HTTP}/api/sync/register/  (X-Achernar-Relay-Token)
       <- { daemon_id, gateway_url }
daemon -> WS {ACHERNAR_CLOUD_WS}/ws/gateway/{daemon_id}/?token=...
       -> HELLO / HEARTBEAT(10s) / TELEMETRY(1s)
       <- COMMAND ; -> COMMAND_RESULT
```

URL normalization on both ends matches (trailing-slash handling, `/api` and `/ws` prefixes). Token check is symmetric (`ACHERNAR_RELAY_DEV_TOKEN`).

# 5 Issues Found and Fixes Implemented

| # | Sev. | Issue | Disposition |
|---|---|---|---|
| 1 | High | `CACHES` never defined; relay result handoff rides default per-process LocMemCache — breaks under multi-process/scaled ASGI | Fixed |
| 2 | High | Frontend split-brain: duplicate `src/api` vs `src/lib/api`, two env + two hooks; `Dashboard` imported both | Fixed |
| 3 | High | `.gitignore lib/` excludes `achernar-frontend/src/lib` — transport switch untracked, breaks fresh clone/CI | Fixed |
| 4 | Med | Tauri `dev` had no `beforeDevCommand`; webview pointed at an unstarted :5173 | Fixed |

| # | Sev. | Issue | Disposition |
|---|---|---|---|
| 5 | Low | Generic product naming (window/title "Tauri + React") | **Fixed** |
| 6 | Med | OLS drivers vendored twice (≈9.6 GB committed) | *Roadmap* |
| 7 | Low | `/api/v1/focus/*` stubs; `ai/analyze` dead helper; `config.rs` unused by `main.rs` | *Roadmap* |

Table 3: Issue register. Items 1–5 were corrected and verified this pass.

### 5.1 Fix 1 — Cache backend for the relay handoff (High)

`backend/settings/base.py`. The browser command path depends on the ASGI WebSocket consumer and the synchronous DRF view exchanging a result through `django.core.cache`. With the default LocMemCache this works for a single process but *silently* fails when more than one process/replica runs (the polling worker never sees the writing worker's result; every command returns "pending"). We back the cache with Redis whenever `REDIS_URL` is set, mirroring the existing `CHANNEL_LAYERS` logic.

```
if os.getenv('REDIS_URL'):
    CACHES = {'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': os.getenv('REDIS_URL')}}
else:
    CACHES = {'default': {
        'BACKEND': 'django.core.cache.backends.locmem.LocMemCache',
        'LOCATION': 'achernar-relay'}}
```

### 5.2 Fix 2 — Consolidate the frontend API layer (High)

We made the top-level `src/` canonical and kept the *better* implementation of each duplicated module: the superset `src/api/daemon.ts` (with `relayCommand`, `cameraStreamWsUrl`, `startCameraStream`) and the *transport-aware* `src/hooks/useTelemetrySocket.ts` (which skips the direct daemon socket in browser mode instead of throwing). Imports in `MjpegLiveView.tsx` and `Dashboard.tsx` were repointed off `../lib/...`, and the stale `src/lib/api`, `src/lib/env.ts`, `src/lib/hooks` were deleted. `src/lib/runtime.ts` remains as the single shared util. No live caller used the methods that differed between the copies, so behaviour is preserved.

### 5.3 Fix 3 — Track the frontend `src/lib` directory (High)

`.gitignore`. The Python-oriented `lib/` pattern matched `achernar-frontend/src/lib`, so the transport-switch code was never committed — it builds on the author's machine but is absent from a clean clone. Added scoped negations:

```
!achernar-frontend/src/lib/
!achernar-frontend/src/lib/**
```

`node_modules/**/lib` remains correctly ignored.

### 5.4 Fix 4 — Tauri hosts the web app in development (Med)

`src-tauri/tauri.conf.json`. Added `"beforeDevCommand": "pnpm -dir ../achernar-frontend dev"` so `tauri dev` boots the real frontend on :5173 (which `devUrl` already targets). With the

existing sidecar manager, the desktop now starts the daemon, serves the frontend, and the frontend detects Tauri and talks to the daemon directly — a complete local loop.

### 5.5 Fix 5 — Product naming (Low)

Window title and `index.html` title set to "Achernar" (from the default "Tauri + React").

## 6 Verification

| Check | Result |
|---|---|
| Frontend typecheck | **Pass** — `tsc -noEmit -p tsconfig.app.json` exits 0 under `strict`, `noUnusedLocals` |
| Backend cache config | **Pass** — both backend dotted paths import in Django 4.2.9; LocMem branch round-trips set/get |
| Settings syntax | **Pass** — `py_compile` on base/dev/prod |
| Tauri config | **Pass** — `tauri.conf.json` parses as valid JSON |
| `.gitignore` fix | **Pass** — `git status` now lists `src/lib/` as trackable; `node_modules` lib still ignored |
| Daemon build | **Prior** — debug+release binaries present; no Rust changed this pass |

Table 4: Verification performed this pass.

**Not yet run (deferred, see roadmap):** a full `cargo build` of the daemon (heavy C++/ONNX toolchain), a Django `manage.py check` against the real virtualenv (the committed venv targets host Python 3.14, unavailable in the audit sandbox), and a live three-process smoke test.

## 7 Roadmap to a Working App

### Phase 1 — Make it run end-to-end locally (days)

- Add `achernar-frontend/.env` with `VITE_DAEMON_URL`, `VITE_CLOUD_API`, `VITE_DAEMON_WS`; document `VITE_HARDWARE_TRANSPORT` override.
- Smoke-test both loops: `start_local.sh` (browser→Django→daemon relay) and `pnpm tauri dev` (desktop→daemon direct), using the `sim` camera driver.
- Decide the `ai/analyze` contract: implement `/api/v1/ai/analyze` in the daemon or remove the dead frontend helper.
- Wire `main.rs` to `AchernarConfig::load()` so `achernar.toml` actually configures the daemon (today only env vars are read).

### Phase 2 — Repository hygiene (days)

- De-duplicate OpenLiveStacker: keep one vendored copy (git submodule *or* a single tracked tree) and reference it from the daemon; move large binaries to Git LFS. This reclaims ≈4.8 GB and removes drift between the two copies.

- Remove the dead Tauri scaffold (`src/App.jsx`, `main.jsx`, `assets/react.svg`) now that the shell hosts the frontend.
- Add CI: `tsc`, `cargo check` (daemon + desktop), `manage.py check`. This would have caught Fixes 1–3 automatically.

### Phase 3 — Spec features (weeks)

- Implement the autofocus control loop behind `/api/v1/focus/*` (`control_supervisor` → `action_arbiter` → `safety_guard` → `focus_agent`).
- Ship the edge ONNX models (FQE / COC / TED / background-SNR) and load them via the model registry.
- Implement session-frame upload (`apps/sync/upload-session` is a 501 stub) and the telemetry "digital twin" snapshot history.

### Phase 4 — Hardening for production (weeks)

- Replace the shared dev relay token + `AllowAny` equipment endpoints with real per-user auth and per-daemon ownership.
- Set a real CSP (currently `null`), enable prod security settings, and add observability (structured logs, health/metrics) across all tiers.

---

End of report — Achernar end-to-end wiring audit, June 13, 2026.